

Relatório de Complexidade (Shell, Merge e Heap Sort)

Vitor K. Costa, Fernando Nunes e Arthur Abreu
Universidade do Vale do Itajaí (Univali) – Campus KobraSol

16 de Junho de 2026

Resumo

Este relatório analisa o desempenho prático de três algoritmos de ordenação: *Shell Sort*, *Merge Sort* e *Heap Sort*. A avaliação foi feita medindo duas operações básicas: o número de Comparações $C(n)$ e a quantidade de Movimentações de dados $M(n)$. Os testes foram executados em diferentes tamanhos de entrada e em três situações: dados ordenados (crescente), dados invertidos (decrescente) e dados embaralhados (aleatório). Os resultados são apresentados em valores relativos percentuais para isolar qual algoritmo exige maior custo computacional em cada cenário.

1 Introdução

O estudo do desempenho de algoritmos de ordenação vai muito além de apenas se basear na notação BigO. Na prática, é essencial medir as operações estruturais que mais consomem processamento: as comparações $C(n)$ (avaliar chaves lógicas) e as movimentações $M(n)$ (troca física de elementos na memória).

- **Shell Sort:** É uma melhoria do *Insertion Sort* desenhada para realizar trocas de elementos distantes. A eficiência do método depende estritamente da sequência de intervalos (*gaps*) adotada, o que pode causar degradação operacional severa dependendo da distribuição da entropia no vetor original.
- **Merge Sort:** Otimizado pelo princípio de *divisão e conquista*, este método particiona recursivamente a estrutura de dados. Ele garante matematicamente o limite de tempo $\Theta(n \log n)$ em todas as permutações de entrada. Sua limitação física reside na incapacidade de operar *in-place*, exigindo a alocação de vetores auxiliares.
- **Heap Sort:** Estruturado em torno da lógica de *max-heap* (uma árvore binária onde nós pais mantêm valores superiores aos filhos). Possui limite garantido de $\mathcal{O}(n \log n)$ e executa a ordenação de forma *in-place*. Contudo, a necessidade de restaurar constantemente as propriedades da árvore (*heapify*) exige repetições exaustivas de comparações lógicas e trocas de nós.

2 Metodologia

As simulações leram arquivos de texto populados com até 10.000 números inteiros. A coleta da variável $C(n)$ foi injetada no interior das estruturas condicionais e de repetição lógicas (e.g., $L[i] \leq R[j]$). A coleta da variável $M(n)$ foi aplicada sobre qualquer cópia, atribuição cruzada ou realocação em memória temporária.

Para estabelecer um parâmetro de comparação justo entre métodos com complexidades distintas, calculou-se o **Valor Relativo**. Em cada simulação, a métrica mais ineficiente (o pior valor absoluto) converte-se na base (100%). O rendimento percentual dos demais métodos foi extraído pela fórmula:

$$V_{relativo} = \left(\frac{\text{Valor Atual}}{\text{Valor Máximo do Cenário}} \right) \times 100 \quad (1)$$

3 Resultados Empíricos

As tabelas expõem o percentual de ineficiência alcançado por cada bloco estrutural do código nas diferentes configurações de dados.

3.1 Cenário 1: Dados em Ordem Aleatória

Nesta configuração de dados embaralhados, o teste evita instigar os piores cenários matemáticos isolados e aproxima-se do uso convencional no mundo real.

Tabela 1: Valores Relativos (%) - Entrada Aleatória

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	91,7	51,8	100,0	87,0	77,9	100,0
500	87,0	51,6	100,0	84,0	73,9	100,0
1000	90,5	51,4	100,0	87,0	73,0	100,0
5000	100,0	48,8	95,1	99,7	72,1	100,0
10000	100,0	45,7	89,4	100,0	68,8	96,0

3.2 Cenário 2: Dados em Ordem Crescente (Melhor Caso)

O vetor entra em estado de pré-ordenação correta. O objetivo primário deste teste é atestar a capacidade da mecânica do algoritmo de reconhecer a ordenação implícita e pular processos redundantes.

Tabela 2: Valores Relativos (%) - Entrada Crescente

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	46,5	32,9	100,0	52,4	70,0	100,0
500	45,2	29,3	100,0	53,7	68,7	100,0
1000	45,5	28,7	100,0	55,0	68,5	100,0
5000	49,1	28,5	100,0	60,2	67,6	100,0
10000	49,1	28,2	100,0	60,6	67,5	100,0

3.3 Cenário 3: Dados em Ordem Decrescente (Pior Caso)

A estrutura entra no sistema em ordem estritamente reversa, exigindo distâncias máximas de trânsito em memória para a recolocação dos índices.

Tabela 3: Valores Relativos (%) - Entrada Decrescente

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	70,8	33,5	100,0	81,8	86,8	100,0
500	73,0	31,6	100,0	82,6	81,4	100,0
1000	73,4	30,9	100,0	83,0	80,0	100,0
5000	76,3	28,9	100,0	86,6	77,1	100,0
10000	76,1	28,5	100,0	86,4	76,3	100,0

4 Conclusões

A análise dos dados gerados na simulação possibilita traçar três deduções consolidadas a respeito da arquitetura computacional dos métodos:

1. **A Quebra de Ciclos no Heap Sort:** O *Heap Sort* demonstrou ser o grande gargalo operacional perante dados estruturados. Nos Cenários 2 (Crescente) e 3 (Decrescente), ele cravou a marca de 100% de ineficiência tanto em movimentações quanto em comparações absolutas. O ato de transmutar um vetor inteiro na topologia de um max-heap corrompe o alinhamento orgânico da lista original. Isso exige um número brutal de checagens lógicas (rebaixamentos e extrações) e trocas sequenciais na memória, tornando-o o algoritmo de maior custo operacional puro da bateria.
2. **O Baixo Custo do Merge Sort:** Em oposição aos modelos teóricos superficiais que costumam penalizar o *Merge Sort* pelas suas cópias em memória, a medição bruta provou o contrário. O algoritmo realizou o **menor número** absoluto de operações $C(n)$ e $M(n)$ na grande maioria dos testes, mantendo suas métricas estabilizadas na faixa dos 28% a 51% relativos. A quantidade de movimentos exigidos para copiar e reconstruir os subvetores ($2n \log n$) foi empiricamente inferior à soma de swaps contínuos executados pelo *Heap Sort*.

3. **A Degradação do Shell Sort:** O *Shell Sort* suportou os cenários pré-ordenados sem entrar no colapso esperado da complexidade $\mathcal{O}(n^2)$. No entanto, na entrada puramente Aleatória (Tabela 1), a mecânica de seus intervalos demonstrou falta de escalabilidade. A partir de $N = 5000$, as rotações em memória e os testes falhos superaram o overhead do *Heap Sort*, levando o *Shell Sort* a assumir a posição de teto máximo (100%) em custo operacional.

Referências

- [1] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.